- ***1*** Creates a second model by making an identical copy of the original Q-network model
- ***2*** Copies the parameters of the original model
- ***3*** Synchronizes the frequency parameter; every 50 steps we will copy the parameters of model into model2
- ***4*** (Code omitted) Uses the same other settings as in listing 3.5

The target network is simply a lagged copy of the main DQN. Each PyTorch model has a `state_dict()` method that returns all of its parameters organized in a dictionary. We use Python's built-in copy module to duplicate the PyTorch model data structure, and then we use the `load_state_dict` method on `model2` to ensure that it has copied the parameters of the main DQN.

Next we include the full training loop, which is mostly the same as listing 3.5 except that we use `model2` when computing the maximum Q value for the next state. We also include a couple of lines of code to copy the parameters from the main model to `model2` every 50 iterations.

Listing 3.8. DQN with experience replay and target network

```
from collections import deque
epochs = 5000
losses = []
mem_size = 1000
batch_size = 200
replay = deque(maxlen=mem_size)
max_moves = 50
h = 0
sync_freq = 500                                          1
j=0
for i in range(epochs):
    game = Gridworld(size=4, mode='random')
    state1_ = game.board.render_np().reshape(1,64) +
np.random.rand(1,64)/100.0
    state1 = torch.from_numpy(state1_).float()
    status = 1
    mov = 0
    while(status == 1):
        j+=1
        mov += 1
        qval = model(state1)
        qval_ = qval.data.numpy()
```